Line-by-Line Mathematical Guide for HAETAE_Security.ec

Generated HAETAE EasyCrypt proof guide

2026-05-11

File Role

Top-level correctness and EUF-CMA theorem composition.

How To Read This Script

Read this file as an inequality-composition script. The local proof steps mostly apply previously checked game-hop and hardness bounds, then normalize the final real-valued bound.

Line-by-Line Mathematical Reading

Each row preserves one EasyCrypt source line and gives the intended mathematical or proof-script reading. Blank rows are kept because they mark proof structure in long EasyCrypt developments.

Line	EasyCrypt source	Mathematical reading
1	`require import AllCore Real Distr StdOrder.`	Imports AllCore Real Distr StdOrder. Mathematically, this exposes earlier carrier sets, operations, games, and lemmas that the current file may cite.
2	`require import Sig_ROM HAETAE_Scheme HAETAE_Algebra HAETAE_Distributions.`	Imports Sig_ROM HAETAE_Scheme HAETAE_Algebra HAETAE_Distributions. Mathematically, this exposes earlier carrier sets, operations, games, and lemmas that the current file may cite.
3	`require import HAETAE_Reductions HAETAE_Assumptions.`	Imports HAETAE_Reductions HAETAE_Assumptions. Mathematically, this exposes earlier carrier sets, operations, games, and lemmas that the current file may cite.
4	`require import HAETAE_Rejection.`	Imports HAETAE_Rejection. Mathematically, this exposes earlier carrier sets, operations, games, and lemmas that the current file may cite.
5	`require import HAETAE_Transcript HAETAE_ROM_Programming.`	Imports HAETAE_Transcript HAETAE_ROM_Programming. Mathematically, this exposes earlier carrier sets, operations, games, and lemmas that the current file may cite.
6	`require import HAETAE_HopGames.`	Imports HAETAE_HopGames. Mathematically, this exposes earlier carrier sets, operations, games, and lemmas that the current file may cite.
7	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8	`theory HAETAE_Security.`	Begins the namespace HAETAE_Security, grouping the declarations as one checked theory.
9	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
10	`import HAETAE_Algebra.`	Opens HAETAE_Algebra so its names can be used without qualification in the following proof.
11	`import HAETAE_Distributions.`	Opens HAETAE_Distributions so its names can be used without qualification in the following proof.
12	`import HAETAE_Reductions.`	Opens HAETAE_Reductions so its names can be used without qualification in the following proof.

Line	EasyCrypt source	Mathematical reading
13	`import HAETAE_Assumptions.`	Opens HAETAE_Assumptions so its names can be used without qualification in the following proof.
14	`import HAETAE_Rejection.`	Opens HAETAE_Rejection so its names can be used without qualification in the following proof.
15	`import HAETAE_Transcript.`	Opens HAETAE_Transcript so its names can be used without qualification in the following proof.
16	`import HAETAE_ROM_Programming.`	Opens HAETAE_ROM_Programming so its names can be used without qualification in the following proof.
17	`import HAETAE_HopGames.`	Opens HAETAE_HopGames so its names can be used without qualification in the following proof.
18	`import HAETAE_Scheme.`	Opens HAETAE_Scheme so its names can be used without qualification in the following proof.
19	`import RealOrder.`	Opens RealOrder so its names can be used without qualification in the following proof.
20	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
21	`op correctness_obligation =`	Defines operation correctness_obligation, a total mathematical function or abbreviation used by later declarations.
22	`forall (sd : seed) (coins : random_coins)`	Part of a universal mathematical condition that must hold for all listed values.
23	`(m : message) (ctx : context),`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
24	`verify_internal haetae_mode (keygen_internal haetae_mode sd).‘1 m ctx`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
25	`(sign_internal haetae_mode`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
26	`(keygen_internal haetae_mode sd).‘2 m ctx coins).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
27	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
28	`lemma correctness_obligation_holds : correctness_obligation.`	States checked lemma correctness_obligation_holds. The following proof script must derive this proposition from prior definitions and lemmas.
29	`proof.`	Begins the proof script for the preceding proposition.
30	`rewrite /correctness_obligation /keygen_internal.`	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
31	`move=> sd coins m ctx.`	Introduces universally quantified variables or hypotheses into the proof context.
32	`by apply verify_internal_sign_internal.`	Discharges the current goal in one step using the cited simplification, lemma, or automation.
33	`qed.`	Ends the proof; EasyCrypt has accepted all remaining obligations.
34	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
35	`section.`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
36	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
37	`declare module H <: SIG.Oracle.`	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
38	`declare module A <: SIG.CORR_ADV {-H, -HAETAE}.`	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
39	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
40	`lemma correctness_perfect &m :`	States checked lemma correctness_perfect. The following proof script must derive this proposition from prior definitions and lemmas.
41	`Pr[SIG.Correctness(H, HAETAE, A).main() @ &m : res] = 0%r.`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
42	`proof.`	Begins the proof script for the preceding proposition.
43	`byphoare => //.`	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
44	`hoare.`	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
45	`proc.`	Enters procedure-body proof mode for a module procedure.
46	`inline *.`	Unfolds a procedure body so the proof can reason about its concrete commands.
47	`wp.`	Applies weakest-precondition reasoning to a program fragment.
48	`call(_: true).`	Uses a procedure specification or adversary call rule inside a program logic proof.

Line	EasyCrypt source	Mathematical reading
49	`wp.`	Applies weakest-precondition reasoning to a program fragment.
50	`call(_: true).`	Uses a procedure specification or adversary call rule inside a program logic proof.
51	`wp.`	Applies weakest-precondition reasoning to a program fragment.
52	`call(_: true).`	Uses a procedure specification or adversary call rule inside a program logic proof.
53	`wp.`	Applies weakest-precondition reasoning to a program fragment.
54	`rnd.`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
55	`wp.`	Applies weakest-precondition reasoning to a program fragment.
56	`call(_: true ==> true).`	Uses a procedure specification or adversary call rule inside a program logic proof.
57	`+ by conseq (: _ ==> true).`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
58	`wp.`	Applies weakest-precondition reasoning to a program fragment.
59	`call(_: true).`	Uses a procedure specification or adversary call rule inside a program logic proof.
60	`wp.`	Applies weakest-precondition reasoning to a program fragment.
61	`rnd.`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
62	`call(_: true).`	Uses a procedure specification or adversary call rule inside a program logic proof.
63	`auto => /> sd _ result coins _.`	Uses EasyCrypt automation for routine logical, arithmetic, or definitional obligations.
64	`move=> result0.`	Introduces universally quantified variables or hypotheses into the proof context.
65	`rewrite /keygen_internal /verify_internal /=.`	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
66	`split.`	Splits a conjunction or structured goal into independent proof obligations.
67	`+ by apply valid_signature_sign_internal.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
68	`by apply verify_norm_ok_current.`	Discharges the current goal in one step using the cited simplification, lemma, or automation.
69	`qed.`	Ends the proof; EasyCrypt has accepted all remaining obligations.
70	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
71	`end section.`	Closes the current section, discharging local module and variable scope.
72	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
73	`section NoSigningSecurity.`	Starts section NoSigningSecurity, a scoped region for related declarations and hypotheses.
74	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
75	`declare module H <: SIG.Oracle {-SIG.EUF_CMA}.`	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
76	`declare module N <: SIG.NMA_Adversary {-H, -HAETAE, -SIG.EUF_CMA}.`	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
77	`declare module Bmlwe <: MLWE_Reduction.`	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
78	`declare module Bbimodal <: BimodalSelfTargetMSIS_Reduction.`	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
79	`declare module Bsis <: ModuleSIS_Reduction.`	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
80	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
81	`lemma euf_cma_security_no_signing &m :`	States checked lemma euf_cma_security_no_signing. The following proof script must derive this proposition from prior definitions and lemmas.
82	`Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m : res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
83	`Pr[MLWE_Game(Bmlwe).main() @ &m : res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
84	`Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m : res] =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
85	`Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m : res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
86	`Pr[ModuleSIS_Game(Bsis).main() @ &m : res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.

Line	EasyCrypt source	Mathematical reading
87	`bimodal_to_msis_reduction_loss_term =>`	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
88	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] <= mlwe_hardness_term =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
89	`Pr[ModuleSIS_Game(Bsis).main() @ &m :  res] <= module_sis_hardness_term =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
90	`Pr[SIG.EUF_CMA(H, HAETAE, SIG.NMA_As_CMA(N)).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
91	`haetae_euf_bound.`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
92	`proof.`	Begins the proof script for the preceding proposition.
93	`move=> nma_hop bimodal_hop mlwe_bound msis_bound.`	Introduces universally quantified variables or hypotheses into the proof context.
94	`rewrite (SIG.nma_as_cma_exact H HAETAE N &m).`	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
95	`rewrite haetae_euf_bound_rejection_groupedE.`	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
96	`have h_nma_msis :`	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
97	`Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
98	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
99	`(Pr[ModuleSIS_Game(Bsis).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
100	`bimodal_to_msis_reduction_loss_term).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
101	`apply (ler_trans`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
102	`(Pr[MLWE_Game(Bmlwe).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
103	`Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m :  res])).`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
104	`+ by apply nma_hop.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
105	`apply ler_add.`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
106	`+ by rewrite lerr.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
107	`+ by apply bimodal_hop.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
108	`apply (ler_trans`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
109	`(Pr[MLWE_Game(Bmlwe).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
110	`(Pr[ModuleSIS_Game(Bsis).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
111	`bimodal_to_msis_reduction_loss_term))).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
112	`+ by apply h_nma_msis.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
113	`apply (ler_trans`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
114	`(mlwe_hardness_term +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
115	`(module_sis_hardness_term + bimodal_to_msis_reduction_loss_term))).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
116	`+ apply ler_add.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
117	`+ by apply mlwe_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
118	`apply ler_add.`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
119	`+ by apply msis_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
120	`+ by rewrite lerr.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.

Line	EasyCrypt source	Mathematical reading
121	`by rewrite ler_addl; apply (rejection_bound_parts_nonnegative`	Discharges the current goal in one step using the cited simplification, lemma, or automation.
122	`rejection_sampling_bound_obligation_holds).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
123	`qed.`	Ends the proof; EasyCrypt has accepted all remaining obligations.
124	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
125	`end section NoSigningSecurity.`	Closes the current section, discharging local module and variable scope.
126	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
127	`section.`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
128	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
129	`declare module H <:  SIG.Oracle.`	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
130	`declare module A <:  SIG.Adversary {-H, -HAETAE}.`	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
131	`declare module N <:  SIG.NMA_Adversary {-H, -HAETAE}.`	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
132	`declare module Bmlwe <:  MLWE_Reduction.`	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
133	`declare module Bbimodal <:  BimodalSelfTargetMSIS_Reduction.`	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
134	`declare module Bsis <:  ModuleSIS_Reduction.`	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
135	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
136	`lemma euf_cma_security &m :`	States checked lemma euf_cma_security. The following proof script must derive this proposition from prior definitions and lemmas.
137	`Pr[SIG.EUF_CMA(H, HAETAE, A).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
138	`Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
139	`(fs_with_aborts_reprogramming_term +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
140	`fs_with_aborts_min_entropy_term) =>`	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
141	`Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
142	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
143	`Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m :  res] =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
144	`Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
145	`Pr[ModuleSIS_Game(Bsis).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
146	`bimodal_to_msis_reduction_loss_term =>`	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
147	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] <= mlwe_hardness_term =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
148	`Pr[ModuleSIS_Game(Bsis).main() @ &m :  res] <= module_sis_hardness_term =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
149	`Pr[SIG.EUF_CMA(H, HAETAE, A).main() @ &m :  res] <= haetae_euf_bound.`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
150	`proof.`	Begins the proof script for the preceding proposition.
151	`move=> fs_hop nma_hop bimodal_hop mlwe_bound msis_bound.`	Introduces universally quantified variables or hypotheses into the proof context.
152	`have h_nma_msis :`	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
153	`Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
154	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
155	`(Pr[ModuleSIS_Game(Bsis).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.

Line	EasyCrypt source	Mathematical reading
156	`bimodal_to_msis_reduction_loss_term).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
157	`apply (ler_trans`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
158	`(Pr[MLWE_Game(Bmlwe).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
159	`Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m :  res])).`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
160	`+ by apply nma_hop.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
161	`apply ler_add.`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
162	`+ by rewrite lerr.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
163	`+ by apply bimodal_hop.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
164	`rewrite haetae_euf_bound_groupedE.`	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
165	`apply (ler_trans`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
166	`((Pr[MLWE_Game(Bmlwe).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
167	`(Pr[ModuleSIS_Game(Bsis).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
168	`bimodal_to_msis_reduction_loss_term)) +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
169	`(fs_with_aborts_reprogramming_term +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
170	`fs_with_aborts_min_entropy_term))).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
171	`+ apply (ler_trans`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
172	`(Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
173	`(fs_with_aborts_reprogramming_term +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
174	`fs_with_aborts_min_entropy_term))).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
175	`+ by apply fs_hop.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
176	`apply ler_add.`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
177	`+ by apply h_nma_msis.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
178	`+ by rewrite lerr.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
179	`apply ler_add.`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
180	`+ apply ler_add.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
181	`+ by apply mlwe_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
182	`apply ler_add.`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
183	`+ by apply msis_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
184	`+ by rewrite lerr.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
185	`+ by rewrite lerr.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
186	`qed.`	Ends the proof; EasyCrypt has accepted all remaining obligations.
187	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
188	`lemma euf_cma_security_with_hop_interfaces &m :`	States checked lemma euf_cma_security_with_hop_interfaces. The following proof script must derive this proposition from prior definitions and lemmas.

Line	EasyCrypt source	Mathematical reading
189	`fs_with_aborts_interfaces_ready haetae_mode =>`	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
190	`Pr[SIG.EUF_CMA(H, HAETAE, A).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
191	`Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
192	`(fs_with_aborts_reprogramming_term +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
193	`fs_with_aborts_min_entropy_term) =>`	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
194	`Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
195	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
196	`Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m :  res] =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
197	`Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
198	`Pr[ModuleSIS_Game(Bsis).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
199	`bimodal_to_msis_reduction_loss_term =>`	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
200	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] <= mlwe_hardness_term =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
201	`Pr[ModuleSIS_Game(Bsis).main() @ &m :  res] <= module_sis_hardness_term =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
202	`Pr[SIG.EUF_CMA(H, HAETAE, A).main() @ &m :  res] <= haetae_euf_bound.`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
203	`proof.`	Begins the proof script for the preceding proposition.
204	`move=> _ fs_hop nma_hop bimodal_hop mlwe_bound msis_bound.`	Introduces universally quantified variables or hypotheses into the proof context.
205	`by apply (euf_cma_security &m fs_hop nma_hop bimodal_hop mlwe_bound`	Discharges the current goal in one step using the cited simplification, lemma, or automation.
206	`msis_bound).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
207	`qed.`	Ends the proof; EasyCrypt has accepted all remaining obligations.
208	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
209	`end section.`	Closes the current section, discharging local module and variable scope.
210	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
211	`section FullReductionWithSimulatorHop.`	Starts section FullReductionWithSimulatorHop, a scoped region for related declarations and hypotheses.
212	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
213	`declare module H <:  SIG.Oracle {-SIG.EUF_CMA, -EUF_CMA_SimulatedSign,`	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
214	`-RealSigningSimulator}.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
215	`declare module A <:  SIG.Adversary {-H, -HAETAE, -SIG.EUF_CMA,`	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
216	`-EUF_CMA_SimulatedSign,`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
217	`-RealSigningSimulator}.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
218	`declare module N <:  SIG.NMA_Adversary {-H, -HAETAE}.`	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
219	`declare module Bmlwe <:  MLWE_Reduction.`	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
220	`declare module Bbimodal <:  BimodalSelfTargetMSIS_Reduction.`	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
221	`declare module Bsis <:  ModuleSIS_Reduction.`	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
222	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
223	`lemma euf_cma_security_from_simulated_hop &m :`	States checked lemma euf_cma_security_from_simulated_hop. The following proof script must derive this proposition from prior definitions and lemmas.
224	`fs_with_aborts_interfaces_ready haetae_mode =>`	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
225	`Pr[EUF_CMA_SimulatedSign(H, HAETAE, A, RealSigningSimulator(HAETAE)).main()`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
226	`@ &m :  res] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
227	`Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
228	`(fs_with_aborts_reprogramming_term +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
229	`fs_with_aborts_min_entropy_term) =>`	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
230	`Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
231	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
232	`Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m :  res] =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
233	`Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
234	`Pr[ModuleSIS_Game(Bsis).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
235	`bimodal_to_msis_reduction_loss_term =>`	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
236	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] <= mlwe_hardness_term =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
237	`Pr[ModuleSIS_Game(Bsis).main() @ &m :  res] <= module_sis_hardness_term =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
238	`Pr[SIG.EUF_CMA(H, HAETAE, A).main() @ &m :  res] <= haetae_euf_bound.`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
239	`proof.`	Begins the proof script for the preceding proposition.
240	`move=> iface sim_hop nma_hop bimodal_hop mlwe_bound msis_bound.`	Introduces universally quantified variables or hypotheses into the proof context.
241	`rewrite (real_signing_simulator_exact H HAETAE A &m).`	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
242	`have h_nma_msis :`	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
243	`Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
244	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
245	`(Pr[ModuleSIS_Game(Bsis).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
246	`bimodal_to_msis_reduction_loss_term).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
247	`apply (ler_trans`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
248	`(Pr[MLWE_Game(Bmlwe).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
249	`Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m :  res])).`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
250	`+ by apply nma_hop.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
251	`apply ler_add.`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
252	`+ by rewrite lerr.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
253	`+ by apply bimodal_hop.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
254	`rewrite haetae_euf_bound_groupedE.`	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
255	`apply (ler_trans`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.

Line	EasyCrypt source	Mathematical reading
256	`((Pr[MLWE_Game(Bmlwe).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
257	`(Pr[ModuleSIS_Game(Bsis).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
258	`bimodal_to_msis_reduction_loss_term)) +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
259	`(fs_with_aborts_reprogramming_term +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
260	`fs_with_aborts_min_entropy_term))).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
261	`+ apply (ler_trans`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
262	`(Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
263	`(fs_with_aborts_reprogramming_term +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
264	`fs_with_aborts_min_entropy_term))).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
265	`+ by apply sim_hop.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
266	`apply ler_add.`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
267	`+ by apply h_nma_msis.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
268	`+ by rewrite lerr.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
269	`apply ler_add.`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
270	`+ apply ler_add.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
271	`+ by apply mlwe_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
272	`apply ler_add.`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
273	`+ by apply msis_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
274	`+ by rewrite lerr.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
275	`+ by rewrite lerr.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
276	`qed.`	Ends the proof; EasyCrypt has accepted all remaining obligations.
277	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
278	`lemma euf_cma_security_from_explicit_boundaries &m :`	States checked lemma euf_cma_security_from_explicit_boundaries. The following proof script must derive this proposition from prior definitions and lemmas.
279	`Pr[EUF_CMA_SimulatedSign(H, HAETAE, A, RealSigningSimulator(HAETAE)).main()`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
280	`@ &m :  res] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
281	`Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
282	`(fs_with_aborts_reprogramming_term +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
283	`fs_with_aborts_min_entropy_term) =>`	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
284	`Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
285	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
286	`Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m :  res] =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
287	`Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
288	`Pr[ModuleSIS_Game(Bsis).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.

Line	EasyCrypt source	Mathematical reading
289	`bimodal_to_msis_reduction_loss_term =>`	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
290	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] <= mlwe_hardness_term =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
291	`Pr[ModuleSIS_Game(Bsis).main() @ &m :  res] <= module_sis_hardness_term =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
292	`Pr[SIG.EUF_CMA(H, HAETAE, A).main() @ &m :  res] <= haetae_euf_bound.`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
293	`proof.`	Begins the proof script for the preceding proposition.
294	`move=> sim_hop nma_hop bimodal_hop mlwe_bound msis_bound.`	Introduces universally quantified variables or hypotheses into the proof context.
295	`apply (euf_cma_security_from_simulated_hop &m).`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
296	`+ by apply fs_with_aborts_interfaces_ready_holds.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
297	`+ by apply sim_hop.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
298	`+ by apply nma_hop.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
299	`+ by apply bimodal_hop.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
300	`+ by apply mlwe_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
301	`+ by apply msis_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
302	`qed.`	Ends the proof; EasyCrypt has accepted all remaining obligations.
303	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
304	`end section FullReductionWithSimulatorHop.`	Closes the current section, discharging local module and variable scope.
305	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
306	`section FullReductionWithROMTranscriptHop.`	Starts section FullReductionWithROMTranscriptHop, a scoped region for related declarations and hypotheses.
307	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
308	`declare module H <:  SIG.Oracle {-SIG.EUF_CMA,`	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
309	`-EUF_CMA_InternalTranscriptSign,`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
310	`-EUF_CMA_ROMInternalTranscriptSign}.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
311	`declare module A <:  SIG.Adversary {-H, -HAETAE, -SIG.EUF_CMA,`	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
312	`-EUF_CMA_InternalTranscriptSign,`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
313	`-EUF_CMA_ROMInternalTranscriptSign}.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
314	`declare module N <:  SIG.NMA_Adversary {-H, -HAETAE}.`	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
315	`declare module Bmlwe <:  MLWE_Reduction.`	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
316	`declare module Bbimodal <:  BimodalSelfTargetMSIS_Reduction.`	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
317	`declare module Bsis <:  ModuleSIS_Reduction.`	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
318	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
319	`lemma euf_cma_security_from_rom_internal_transcript_hop &m :`	States checked lemma euf_cma_security_from_rom_internal_transcript_hop. The following proof script must derive this proposition from prior definitions and lemmas.
320	`Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
321	`Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
322	`(fs_with_aborts_reprogramming_term +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
323	`fs_with_aborts_min_entropy_term) =>`	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
324	`Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
325	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
326	`Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m :  res] =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
327	`Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
328	`Pr[ModuleSIS_Game(Bsis).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
329	`bimodal_to_msis_reduction_loss_term =>`	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
330	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] <= mlwe_hardness_term =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
331	`Pr[ModuleSIS_Game(Bsis).main() @ &m :  res] <= module_sis_hardness_term =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
332	`Pr[SIG.EUF_CMA(H, HAETAE, A).main() @ &m :  res] <= haetae_euf_bound.`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
333	`proof.`	Begins the proof script for the preceding proposition.
334	`move=> rom_hop nma_hop bimodal_hop mlwe_bound msis_bound.`	Introduces universally quantified variables or hypotheses into the proof context.
335	`apply (euf_cma_security H A N Bmlwe Bbimodal Bsis &m).`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
336	`+ rewrite (haetae_rom_internal_transcript_erasure_exact H A &m).`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
337	`by apply rom_hop.`	Discharges the current goal in one step using the cited simplification, lemma, or automation.
338	`+ by apply nma_hop.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
339	`+ by apply bimodal_hop.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
340	`+ by apply mlwe_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
341	`+ by apply msis_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
342	`qed.`	Ends the proof; EasyCrypt has accepted all remaining obligations.
343	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
344	`lemma rom_internal_transcript_hop_from_clear_site_reduction &m :`	States checked lemma rom_internal_transcript_hop_from_clear_site_reduction. The following proof script must derive this proposition from prior definitions and lemmas.
345	`Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m :`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
346	`res /\`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
347	`transcript_log_signature_programming_sites_clear haetae_mode`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
348	`EUF_CMA_ROMInternalTranscriptSign.transcripts] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
349	`Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m :  res] =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
350	`Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
351	`Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
352	`(fs_with_aborts_reprogramming_term +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
353	`fs_with_aborts_min_entropy_term).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
354	`proof.`	Begins the proof script for the preceding proposition.
355	`move=> clear_reduction.`	Introduces universally quantified variables or hypotheses into the proof context.
356	`have split_success :`	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.

Line	EasyCrypt source	Mathematical reading
357	`Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m :  res] =`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
358	`Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m :`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
359	`res /\`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
360	`transcript_log_signature_programming_sites_clear haetae_mode`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
361	`EUF_CMA_ROMInternalTranscriptSign.transcripts] +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
362	`Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m :`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
363	`res /\`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
364	`!  transcript_log_signature_programming_sites_clear haetae_mode`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
365	`EUF_CMA_ROMInternalTranscriptSign.transcripts].`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
366	`by rewrite Pr[mu_split (transcript_log_signature_programming_sites_clear`	Discharges the current goal in one step using the cited simplification, lemma, or automation.
367	`haetae_mode`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
368	`EUF_CMA_ROMInternalTranscriptSign.transcripts)].`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
369	`rewrite split_success.`	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
370	`apply ler_add.`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
371	`+ by apply clear_reduction.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
372	`by apply (rom_internal_transcript_bad_forgery_bound H A &m).`	Discharges the current goal in one step using the cited simplification, lemma, or automation.
373	`qed.`	Ends the proof; EasyCrypt has accepted all remaining obligations.
374	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
375	`lemma rom_internal_transcript_hop_from_clear_site_counted_reduction &m :`	States checked lemma rom_internal_transcript_hop_from_clear_site_counted_reduction. The following proof script must derive this proposition from prior definitions and lemmas.
376	`Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m :`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
377	`res /\`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
378	`transcript_log_signature_programming_sites_clear haetae_mode`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
379	`EUF_CMA_ROMInternalTranscriptSign.transcripts] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
380	`Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m :  res] =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
381	`Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
382	`Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
383	`counted_rom_programming_loss_term.`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
384	`proof.`	Begins the proof script for the preceding proposition.
385	`move=> clear_reduction.`	Introduces universally quantified variables or hypotheses into the proof context.
386	`have split_success :`	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
387	`Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m :  res] =`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
388	`Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m :`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
389	`res /\`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
390	`transcript_log_signature_programming_sites_clear haetae_mode`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
391	`EUF_CMA_ROMInternalTranscriptSign.transcripts] +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
392	`Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m :`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
393	`res /\`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
394	`!  transcript_log_signature_programming_sites_clear haetae_mode`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
395	`EUF_CMA_ROMInternalTranscriptSign.transcripts].`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
396	`by rewrite Pr[mu_split (transcript_log_signature_programming_sites_clear`	Discharges the current goal in one step using the cited simplification, lemma, or automation.
397	`haetae_mode`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
398	`EUF_CMA_ROMInternalTranscriptSign.transcripts)].`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
399	`rewrite split_success.`	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
400	`apply ler_add.`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
401	`+ by apply clear_reduction.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
402	`by apply (rom_internal_transcript_bad_forgery_counted_bound H A &m).`	Discharges the current goal in one step using the cited simplification, lemma, or automation.
403	`qed.`	Ends the proof; EasyCrypt has accepted all remaining obligations.
404	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
405	`end section FullReductionWithROMTranscriptHop.`	Closes the current section, discharging local module and variable scope.
406	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
407	`section MechanizedBimodalModuleSISLift.`	Starts section MechanizedBimodalModuleSISLift, a scoped region for related declarations and hypotheses.
408	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
409	`declare module H <:  SIG.Oracle {-SIG.EUF_CMA, -EUF_CMA_SimulatedSign,`	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
410	`-RealSigningSimulator}.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
411	`declare module A <:  SIG.Adversary {-H, -HAETAE, -SIG.EUF_CMA,`	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
412	`-EUF_CMA_SimulatedSign,`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
413	`-RealSigningSimulator}.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
414	`declare module N <:  SIG.NMA_Adversary {-H, -HAETAE}.`	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
415	`declare module Bmlwe <:  MLWE_Reduction.`	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
416	`declare module Bbimodal <:  BimodalSelfTargetMSIS_Reduction.`	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
417	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
418	`lemma bimodal_to_module_sis_reduction_bound &m :`	States checked lemma bimodal_to_module_sis_reduction_bound. The following proof script must derive this proposition from prior definitions and lemmas.
419	`Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
420	`Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
421	`@ &m :  res] +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
422	`bimodal_to_msis_reduction_loss_term.`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
423	`proof.`	Begins the proof script for the preceding proposition.

Line	EasyCrypt source	Mathematical reading
424	`rewrite (bimodal_msis_as_module_sis_exact Bbimodal &m).`	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
425	`rewrite bimodal_to_msis_reduction_loss_termE.`	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
426	`have -> :`	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
427	`Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
428	`@ &m :  res] + 0%r =`	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
429	`Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
430	`@ &m :  res] by ring.`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
431	`by rewrite lerr.`	Discharges the current goal in one step using the cited simplification, lemma, or automation.
432	`qed.`	Ends the proof; EasyCrypt has accepted all remaining obligations.
433	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
434	`lemma euf_cma_security_from_mechanized_boundaries &m :`	States checked lemma euf_cma_security_from_mechanized_boundaries. The following proof script must derive this proposition from prior definitions and lemmas.
435	`Pr[EUF_CMA_SimulatedSign(H, HAETAE, A, RealSigningSimulator(HAETAE)).main()`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
436	`@ &m :  res] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
437	`Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
438	`(fs_with_aborts_reprogramming_term +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
439	`fs_with_aborts_min_entropy_term) =>`	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
440	`Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
441	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
442	`Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m :  res] =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
443	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] <= mlwe_hardness_term =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
444	`Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
445	`@ &m :  res] <= module_sis_hardness_term =>`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
446	`Pr[SIG.EUF_CMA(H, HAETAE, A).main() @ &m :  res] <= haetae_euf_bound.`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
447	`proof.`	Begins the proof script for the preceding proposition.
448	`move=> sim_hop nma_hop mlwe_bound msis_bound.`	Introduces universally quantified variables or hypotheses into the proof context.
449	`apply (euf_cma_security_from_simulated_hop`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
450	`H A N Bmlwe Bbimodal (BimodalMSIS_As_ModuleSIS(Bbimodal)) &m).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
451	`+ by apply fs_with_aborts_interfaces_ready_holds.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
452	`+ by apply sim_hop.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
453	`+ by apply nma_hop.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
454	`+ by apply bimodal_to_module_sis_reduction_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
455	`+ by apply mlwe_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
456	`+ by apply msis_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
457	`qed.`	Ends the proof; EasyCrypt has accepted all remaining obligations.

Line	EasyCrypt source	Mathematical reading
458	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
459	`end section MechanizedBimodalModuleSISLift.`	Closes the current section, discharging local module and variable scope.
460	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
461	`section MechanizedROMTranscriptBimodalLift.`	Starts section MechanizedROMTranscriptBimodalLift, a scoped region for related declarations and hypotheses.
462	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
463	`declare module H <:  SIG.Oracle {-SIG.EUF_CMA,`	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
464	`-EUF_CMA_InternalTranscriptSign,`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
465	`-EUF_CMA_ROMInternalTranscriptSign}.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
466	`declare module A <:  SIG.Adversary {-H, -HAETAE, -SIG.EUF_CMA,`	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
467	`-EUF_CMA_InternalTranscriptSign,`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
468	`-EUF_CMA_ROMInternalTranscriptSign}.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
469	`declare module N <:  SIG.NMA_Adversary {-H, -HAETAE}.`	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
470	`declare module Bmlwe <:  MLWE_Reduction.`	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
471	`declare module Bbimodal <:  BimodalSelfTargetMSIS_Reduction.`	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
472	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
473	`lemma euf_cma_security_from_rom_transcript_and_mechanized_bimodal &m :`	States checked lemma euf_cma_security_from_rom_transcript_and_mechanized_bimodal. The following proof script must derive this proposition from prior definitions and lemmas.
474	`Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
475	`Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
476	`(fs_with_aborts_reprogramming_term +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
477	`fs_with_aborts_min_entropy_term) =>`	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
478	`Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
479	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
480	`Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m :  res] =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
481	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] <= mlwe_hardness_term =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
482	`Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
483	`@ &m :  res] <= module_sis_hardness_term =>`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
484	`Pr[SIG.EUF_CMA(H, HAETAE, A).main() @ &m :  res] <= haetae_euf_bound.`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
485	`proof.`	Begins the proof script for the preceding proposition.
486	`move=> rom_hop nma_hop mlwe_bound msis_bound.`	Introduces universally quantified variables or hypotheses into the proof context.
487	`apply (euf_cma_security_from_rom_internal_transcript_hop`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
488	`H A N Bmlwe Bbimodal (BimodalMSIS_As_ModuleSIS(Bbimodal)) &m).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
489	`+ by apply rom_hop.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
490	`+ by apply nma_hop.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.

Line	EasyCrypt source	Mathematical reading
491	`+ by apply (bimodal_to_module_sis_reduction_bound Bbimodal &m).`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
492	`+ by apply mlwe_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
493	`+ by apply msis_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
494	`qed.`	Ends the proof; EasyCrypt has accepted all remaining obligations.
495	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
496	`lemma euf_cma_security_from_rom_transcript_and_counted_bimodal &m :`	States checked lemma euf_cma_security_from_rom_transcript_and_counted_bimodal. The following proof script must derive this proposition from prior definitions and lemmas.
497	`Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
498	`Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
499	`counted_rom_programming_loss_term =>`	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
500	`Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
501	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
502	`Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m :  res] =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
503	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] <= mlwe_hardness_term =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
504	`Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
505	`@ &m :  res] <= module_sis_hardness_term =>`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
506	`Pr[SIG.EUF_CMA(H, HAETAE, A).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
507	`haetae_euf_counted_rom_bound.`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
508	`proof.`	Begins the proof script for the preceding proposition.
509	`move=> rom_hop nma_hop mlwe_bound msis_bound.`	Introduces universally quantified variables or hypotheses into the proof context.
510	`rewrite (haetae_rom_internal_transcript_erasure_exact H A &m).`	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
511	`have h_nma_msis :`	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
512	`Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
513	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
514	`(Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
515	`@ &m :  res] +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
516	`bimodal_to_msis_reduction_loss_term).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
517	`apply (ler_trans`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
518	`(Pr[MLWE_Game(Bmlwe).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
519	`Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m :  res])).`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
520	`+ by apply nma_hop.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
521	`apply ler_add.`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
522	`+ by rewrite lerr.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
523	`+ by apply (bimodal_to_module_sis_reduction_bound Bbimodal &m).`	Solves a proof branch produced by a previous split, transitivity, or case analysis.

Line	EasyCrypt source	Mathematical reading
524	`rewrite haetae_euf_counted_rom_bound_groupedE.`	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
525	`apply (ler_trans`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
526	`((Pr[MLWE_Game(Bmlwe).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
527	`(Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
528	`@ &m :  res] +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
529	`bimodal_to_msis_reduction_loss_term)) +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
530	`counted_rom_programming_loss_term)).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
531	`+ apply (ler_trans`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
532	`(Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
533	`counted_rom_programming_loss_term)).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
534	`+ by apply rom_hop.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
535	`apply ler_add.`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
536	`+ by apply h_nma_msis.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
537	`+ by rewrite lerr.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
538	`apply ler_add.`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
539	`+ apply ler_add.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
540	`+ by apply mlwe_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
541	`apply ler_add.`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
542	`+ by apply msis_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
543	`+ by rewrite lerr.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
544	`+ by rewrite lerr.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
545	`qed.`	Ends the proof; EasyCrypt has accepted all remaining obligations.
546	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
547	`lemma euf_cma_security_from_rom_clear_site_and_mechanized_bimodal &m :`	States checked lemma euf_cma_security_from_rom_clear_site_and_mechanized_bimodal. The following proof script must derive this proposition from prior definitions and lemmas.
548	`Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m :`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
549	`res /\`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
550	`transcript_log_signature_programming_sites_clear haetae_mode`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
551	`EUF_CMA_ROMInternalTranscriptSign.transcripts] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
552	`Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m :  res] =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
553	`Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
554	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
555	`Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m :  res] =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
556	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] <= mlwe_hardness_term =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.

Line	EasyCrypt source	Mathematical reading
557	`Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
558	`@ &m :  res] <= module_sis_hardness_term =>`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
559	`Pr[SIG.EUF_CMA(H, HAETAE, A).main() @ &m :  res] <= haetae_euf_bound.`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
560	`proof.`	Begins the proof script for the preceding proposition.
561	`move=> clear_reduction nma_hop mlwe_bound msis_bound.`	Introduces universally quantified variables or hypotheses into the proof context.
562	`apply (euf_cma_security_from_rom_transcript_and_mechanized_bimodal`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
563	`&m).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
564	`+ by apply (rom_internal_transcript_hop_from_clear_site_reduction`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
565	`H A N &m clear_reduction).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
566	`+ by apply nma_hop.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
567	`+ by apply mlwe_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
568	`+ by apply msis_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
569	`qed.`	Ends the proof; EasyCrypt has accepted all remaining obligations.
570	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
571	`lemma euf_cma_security_from_rom_clear_site_and_counted_bimodal &m :`	States checked lemma euf_cma_security_from_rom_clear_site_and_counted_bimodal. The following proof script must derive this proposition from prior definitions and lemmas.
572	`Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m :`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
573	`res /\`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
574	`transcript_log_signature_programming_sites_clear haetae_mode`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
575	`EUF_CMA_ROMInternalTranscriptSign.transcripts] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
576	`Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m :  res] =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
577	`Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
578	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
579	`Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m :  res] =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
580	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] <= mlwe_hardness_term =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
581	`Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
582	`@ &m :  res] <= module_sis_hardness_term =>`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
583	`Pr[SIG.EUF_CMA(H, HAETAE, A).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
584	`haetae_euf_counted_rom_bound.`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
585	`proof.`	Begins the proof script for the preceding proposition.
586	`move=> clear_reduction nma_hop mlwe_bound msis_bound.`	Introduces universally quantified variables or hypotheses into the proof context.
587	`apply (euf_cma_security_from_rom_transcript_and_counted_bimodal`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
588	`&m).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
589	`+ by apply (rom_internal_transcript_hop_from_clear_site_counted_reduction`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
590	`H A N &m clear_reduction).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
591	`+ by apply nma_hop.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
592	`+ by apply mlwe_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
593	`+ by apply msis_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
594	`qed.`	Ends the proof; EasyCrypt has accepted all remaining obligations.
595	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
596	`end section MechanizedROMTranscriptBimodalLift.`	Closes the current section, discharging local module and variable scope.
597	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
598	`section MechanizedROMTranscriptConstructedNMALift.`	Starts section MechanizedROMTranscriptConstructedNMALift, a scoped region for related declarations and hypotheses.
599	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
600	`declare module H <:  SIG.Oracle {-SIG.EUF_CMA,`	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
601	`-EUF_CMA_InternalTranscriptSign,`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
602	`-EUF_CMA_ROMInternalTranscriptSign,`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
603	`-ROMInternalTranscriptAsNMA,`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
604	`-ROMInternalTranscriptPublicSimAsNMA,`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
605	`-ROMInternalTranscriptPaperSimAsNMA,`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
606	`-ROMInternalTranscriptBudgetedPaperSimAsNMA,`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
607	`-ROMPaperSimSigningSampler,`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
608	`-RealSigningPaperSimSampler,`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
609	`-ROSigningAttemptPaperSimSampler,`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
610	`-HAETAERejectionPaperSimSampler,`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
611	`-ExactHyperballHAETAERejectionPaperSimSampler,`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
612	`-ExactHyperballPaperSimSampler,`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
613	`-ROExactHyperballPaperSimSampler}.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
614	`declare module A <:  SIG.Adversary {-H, -HAETAE, -SIG.EUF_CMA,`	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
615	`-EUF_CMA_InternalTranscriptSign,`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
616	`-EUF_CMA_ROMInternalTranscriptSign,`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
617	`-ROMInternalTranscriptAsNMA,`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
618	`-ROMInternalTranscriptPublicSimAsNMA,`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
619	`-ROMInternalTranscriptPaperSimAsNMA,`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
620	`-ROMInternalTranscriptBudgetedPaperSimAsNMA,`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
621	`-ROMPaperSimSigningSampler,`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
622	`-RealSigningPaperSimSampler,`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
623	`-ROSigningAttemptPaperSimSampler,`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
624	`-HAETAERejectionPaperSimSampler,`	Solves a proof branch produced by a previous split, transitivity, or case analysis.

Line	EasyCrypt source	Mathematical reading
625	`-ExactHyperballHAETAERejectionPaperSimSampler,`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
626	`-ExactHyperballPaperSimSampler,`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
627	`-ROExactHyperballPaperSimSampler}.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
628	`declare module Samp <:  PaperSimSigningSampler {-H, -A,`	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
629	`-ROMInternalTranscriptPaperSimAsNMA}.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
630	`declare module Bmlwe <:  MLWE_Reduction.`	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
631	`declare module Bbimodal <:  BimodalSelfTargetMSIS_Reduction.`	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
632	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
633	`lemma euf_cma_security_from_structural_nma_and_counted_bimodal &m :`	States checked lemma euf_cma_security_from_structural_nma_and_counted_bimodal. The following proof script must derive this proposition from prior definitions and lemmas.
634	`Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptAsNMA(A)).main()`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
635	`@ &m :  res] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
636	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
637	`Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m :  res] =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
638	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] <= mlwe_hardness_term =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
639	`Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
640	`@ &m :  res] <= module_sis_hardness_term =>`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
641	`Pr[SIG.EUF_CMA(H, HAETAE, A).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
642	`haetae_euf_counted_rom_bound.`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
643	`proof.`	Begins the proof script for the preceding proposition.
644	`move=> nma_hop mlwe_bound msis_bound.`	Introduces universally quantified variables or hypotheses into the proof context.
645	`apply (euf_cma_security_from_rom_clear_site_and_counted_bimodal`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
646	`H A (ROMInternalTranscriptAsNMA(A)) Bmlwe Bbimodal &m).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
647	`+ by apply (rom_internal_transcript_clear_site_structural_nma_bound H A &m).`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
648	`+ by apply nma_hop.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
649	`+ by apply mlwe_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
650	`+ by apply msis_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
651	`qed.`	Ends the proof; EasyCrypt has accepted all remaining obligations.
652	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
653	`lemma euf_cma_security_from_public_sim_nma_and_counted_bimodal &m :`	States checked lemma euf_cma_security_from_public_sim_nma_and_counted_bimodal. The following proof script must derive this proposition from prior definitions and lemmas.
654	`Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPublicSimAsNMA(A)).main()`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
655	`@ &m :  res] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
656	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.

Line	EasyCrypt source	Mathematical reading
657	`Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m :  res] =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
658	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] <= mlwe_hardness_term =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
659	`Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
660	`@ &m :  res] <= module_sis_hardness_term =>`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
661	`Pr[SIG.EUF_CMA(H, HAETAE, A).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
662	`haetae_euf_counted_rom_bound.`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
663	`proof.`	Begins the proof script for the preceding proposition.
664	`move=> nma_hop mlwe_bound msis_bound.`	Introduces universally quantified variables or hypotheses into the proof context.
665	`apply (euf_cma_security_from_structural_nma_and_counted_bimodal`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
666	`&m).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
667	`+ rewrite (rom_internal_nma_public_sim_exact H A &m).`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
668	`by apply nma_hop.`	Discharges the current goal in one step using the cited simplification, lemma, or automation.
669	`+ by apply mlwe_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
670	`+ by apply msis_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
671	`qed.`	Ends the proof; EasyCrypt has accepted all remaining obligations.
672	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
673	`lemma euf_cma_security_from_paper_sim_nma_and_counted_bimodal &m :`	States checked lemma euf_cma_security_from_paper_sim_nma_and_counted_bimodal. The following proof script must derive this proposition from prior definitions and lemmas.
674	`Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPublicSimAsNMA(A)).main()`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
675	`@ &m :  res] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
676	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
677	`ROMInternalTranscriptPaperSimAsNMA(A, Samp)).main() @ &m :  res] +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
678	`rejection_sampling_loss_term =>`	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
679	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
680	`ROMInternalTranscriptPaperSimAsNMA(A, Samp)).main() @ &m :  res] +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
681	`rejection_sampling_loss_term <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
682	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
683	`Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m :  res] =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
684	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] <= mlwe_hardness_term =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
685	`Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
686	`@ &m :  res] <= module_sis_hardness_term =>`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
687	`Pr[SIG.EUF_CMA(H, HAETAE, A).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
688	`haetae_euf_counted_rom_bound.`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
689	`proof.`	Begins the proof script for the preceding proposition.
690	`move=> sampling_hop paper_nma_hop mlwe_bound msis_bound.`	Introduces universally quantified variables or hypotheses into the proof context.

Line	EasyCrypt source	Mathematical reading
691	`apply (euf_cma_security_from_public_sim_nma_and_counted_bimodal`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
692	`&m).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
693	`+ apply (ler_trans`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
694	`(Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
695	`ROMInternalTranscriptPaperSimAsNMA(A, Samp)).main() @ &m :  res] +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
696	`rejection_sampling_loss_term)).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
697	`+ by apply (public_sim_to_paper_sim_nma_hop_from_sampling_hop`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
698	`H A Samp &m).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
699	`+ by apply paper_nma_hop.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
700	`+ by apply mlwe_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
701	`+ by apply msis_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
702	`qed.`	Ends the proof; EasyCrypt has accepted all remaining obligations.
703	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
704	`lemma euf_cma_security_from_rom_paper_sim_nma_and_counted_bimodal &m :`	States checked lemma euf_cma_security_from_rom_paper_sim_nma_and_counted_bimodal. The following proof script must derive this proposition from prior definitions and lemmas.
705	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
706	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
707	`(A, ROMPaperSimSigningSampler(H))).main() @ &m :  res] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
708	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
709	`Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m :  res] =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
710	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] <= mlwe_hardness_term =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
711	`Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
712	`@ &m :  res] <= module_sis_hardness_term =>`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
713	`Pr[SIG.EUF_CMA(H, HAETAE, A).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
714	`haetae_euf_counted_rom_bound.`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
715	`proof.`	Begins the proof script for the preceding proposition.
716	`move=> nma_hop mlwe_bound msis_bound.`	Introduces universally quantified variables or hypotheses into the proof context.
717	`apply (euf_cma_security_from_public_sim_nma_and_counted_bimodal`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
718	`&m).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
719	`+ rewrite (rom_internal_nma_public_sim_rom_paper_sim_exact H A &m).`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
720	`by apply nma_hop.`	Discharges the current goal in one step using the cited simplification, lemma, or automation.
721	`+ by apply mlwe_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
722	`+ by apply msis_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
723	`qed.`	Ends the proof; EasyCrypt has accepted all remaining obligations.
724	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
725	`lemma euf_cma_security_from_real_signing_paper_sim_nma_and_counted_bimodal &m :`	States checked lemma euf_cma_security_from_real_signing_paper_sim_nma_and_counted_bimodal. The following proof script must derive this proposition from prior definitions and lemmas.
726	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
727	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
728	`(A, RealSigningPaperSimSampler(H))).main() @ &m :  res] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
729	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
730	`Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m :  res] =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
731	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] <= mlwe_hardness_term =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
732	`Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
733	`@ &m :  res] <= module_sis_hardness_term =>`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
734	`Pr[SIG.EUF_CMA(H, HAETAE, A).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
735	`haetae_euf_counted_rom_bound.`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
736	`proof.`	Begins the proof script for the preceding proposition.
737	`move=> nma_hop mlwe_bound msis_bound.`	Introduces universally quantified variables or hypotheses into the proof context.
738	`apply (euf_cma_security_from_structural_nma_and_counted_bimodal`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
739	`&m).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
740	`+ rewrite (rom_internal_nma_real_signing_paper_sim_exact H A &m).`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
741	`by apply nma_hop.`	Discharges the current goal in one step using the cited simplification, lemma, or automation.
742	`+ by apply mlwe_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
743	`+ by apply msis_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
744	`qed.`	Ends the proof; EasyCrypt has accepted all remaining obligations.
745	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
746	`lemma euf_cma_security_from_haetae_rejection_paper_sim_nma_and_counted_bimodal &m :`	States checked lemma euf_cma_security_from_haetae_rejection_paper_sim_nma_and_counted_bimodal. The following proof script must derive this proposition from prior definitions and lemmas.
747	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
748	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
749	`(A, HAETAERejectionPaperSimSampler(H))).main() @ &m :  res] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
750	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
751	`Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m :  res] =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
752	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] <= mlwe_hardness_term =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
753	`Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
754	`@ &m :  res] <= module_sis_hardness_term =>`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
755	`Pr[SIG.EUF_CMA(H, HAETAE, A).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
756	`haetae_euf_counted_rom_bound.`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
757	`proof.`	Begins the proof script for the preceding proposition.
758	`move=> nma_hop mlwe_bound msis_bound.`	Introduces universally quantified variables or hypotheses into the proof context.
759	`apply (euf_cma_security_from_real_signing_paper_sim_nma_and_counted_bimodal`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
760	`&m).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
761	`+ rewrite (rom_internal_nma_real_signing_haetae_rejection_paper_sim_exact`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
762	`H A &m).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
763	`by apply nma_hop.`	Discharges the current goal in one step using the cited simplification, lemma, or automation.
764	`+ by apply mlwe_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
765	`+ by apply msis_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
766	`qed.`	Ends the proof; EasyCrypt has accepted all remaining obligations.
767	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
768	`lemma exact_hyperball_haetae_rejection_nma_bound_from_paper_sim &m :`	States checked lemma exact_hyperball_haetae_rejection_nma_bound_from_paper_sim. The following proof script must derive this proposition from prior definitions and lemmas.
769	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
770	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
771	`(A, ExactHyperballPaperSimSampler(H))).main() @ &m :  res] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
772	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
773	`Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m :  res] =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
774	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
775	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
776	`(A, ExactHyperballHAETAERejectionPaperSimSampler(H))).main()`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
777	`@ &m :  res] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
778	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
779	`Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m :  res].`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
780	`proof.`	Begins the proof script for the preceding proposition.
781	`move=> nma_hop.`	Introduces universally quantified variables or hypotheses into the proof context.
782	`rewrite (rom_internal_nma_exact_hyperball_rejection_paper_sim_no_fallback_exact`	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
783	`H A &m).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
784	`by apply nma_hop.`	Discharges the current goal in one step using the cited simplification, lemma, or automation.
785	`qed.`	Ends the proof; EasyCrypt has accepted all remaining obligations.
786	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
787	`lemma haetae_rejection_nma_bound_from_exact_hyperball_sampling_hop &m :`	States checked lemma haetae_rejection_nma_bound_from_exact_hyperball_sampling_hop. The following proof script must derive this proposition from prior definitions and lemmas.
788	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
789	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
790	`(A, HAETAERejectionPaperSimSampler(H))).main() @ &m :  res] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
791	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
792	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
793	`(A, ExactHyperballHAETAERejectionPaperSimSampler(H))).main()`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
794	`@ &m :  res] + rejection_sampling_loss_term =>`	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
795	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
796	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
797	`(A, ExactHyperballPaperSimSampler(H))).main() @ &m :  res] +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
798	`rejection_sampling_loss_term <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
799	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
800	`Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m :  res] =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
801	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
802	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
803	`(A, HAETAERejectionPaperSimSampler(H))).main() @ &m :  res] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
804	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
805	`Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m :  res].`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
806	`proof.`	Begins the proof script for the preceding proposition.
807	`move=> sampling_hop exact_nma_hop.`	Introduces universally quantified variables or hypotheses into the proof context.
808	`apply (ler_trans`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
809	`(Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
810	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
811	`(A, ExactHyperballHAETAERejectionPaperSimSampler(H))).main()`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
812	`@ &m :  res] + rejection_sampling_loss_term)).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
813	`+ by apply sampling_hop.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
814	`rewrite (rom_internal_nma_exact_hyperball_rejection_paper_sim_no_fallback_exact`	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
815	`H A &m).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
816	`by apply exact_nma_hop.`	Discharges the current goal in one step using the cited simplification, lemma, or automation.
817	`qed.`	Ends the proof; EasyCrypt has accepted all remaining obligations.
818	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
819	`lemma haetae_rejection_exact_hyperball_sampling_hop_from_real_signing_exact_hyperball_hop &m :`	States checked lemma haetae_rejection_exact_hyperball_sampling_hop_from_real_signing_exact_hyperball_ho The following proof script must derive this proposition from prior definitions and lemmas.
820	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
821	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
822	`(A, RealSigningPaperSimSampler(H))).main() @ &m :  res] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.

Line	EasyCrypt source	Mathematical reading
823	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
824	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
825	`(A, ExactHyperballPaperSimSampler(H))).main() @ &m :  res] +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
826	`rejection_sampling_loss_term =>`	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
827	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
828	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
829	`(A, HAETAERejectionPaperSimSampler(H))).main() @ &m :  res] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
830	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
831	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
832	`(A, ExactHyperballHAETAERejectionPaperSimSampler(H))).main()`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
833	`@ &m :  res] + rejection_sampling_loss_term.`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
834	`proof.`	Begins the proof script for the preceding proposition.
835	`move=> real_exact_hop.`	Introduces universally quantified variables or hypotheses into the proof context.
836	`rewrite -(rom_internal_nma_real_signing_haetae_rejection_paper_sim_exact`	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
837	`H A &m).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
838	`rewrite (rom_internal_nma_exact_hyperball_rejection_paper_sim_no_fallback_exact`	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
839	`H A &m).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
840	`by apply real_exact_hop.`	Discharges the current goal in one step using the cited simplification, lemma, or automation.
841	`qed.`	Ends the proof; EasyCrypt has accepted all remaining obligations.
842	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
843	`lemma haetae_rejection_ro_exact_hyperball_sampling_hop_from_real_signing_ro_exact_hyperball_hop &m :`	States checked lemma haetae_rejection_ro_exact_hyperball_sampling_hop_from_real_signing_ro_exact_hyper The following proof script must derive this proposition from prior definitions and lemmas.
844	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
845	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
846	`(A, RealSigningPaperSimSampler(H))).main() @ &m :  res] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
847	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
848	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
849	`(A, ROExactHyperballPaperSimSampler(H))).main() @ &m :  res] +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
850	`rejection_sampling_loss_term =>`	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
851	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
852	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
853	`(A, HAETAERejectionPaperSimSampler(H))).main() @ &m :  res] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
854	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
855	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
856	`(A, ROExactHyperballPaperSimSampler(H))).main() @ &m :  res] +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
857	`rejection_sampling_loss_term.`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
858	`proof.`	Begins the proof script for the preceding proposition.
859	`move=> real_ro_exact_hop.`	Introduces universally quantified variables or hypotheses into the proof context.
860	`rewrite -(rom_internal_nma_real_signing_haetae_rejection_paper_sim_exact`	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
861	`H A &m).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
862	`by apply real_ro_exact_hop.`	Discharges the current goal in one step using the cited simplification, lemma, or automation.
863	`qed.`	Ends the proof; EasyCrypt has accepted all remaining obligations.
864	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
865	`lemma real_signing_ro_exact_hyperball_hop_from_ro_signing_attempt_hop &m :`	States checked lemma real_signing_ro_exact_hyperball_hop_from_ro_signing_attempt_hop. The following proof script must derive this proposition from prior definitions and lemmas.
866	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
867	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
868	`(A, ROSigningAttemptPaperSimSampler(H))).main() @ &m :  res] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
869	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
870	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
871	`(A, ROExactHyperballPaperSimSampler(H))).main() @ &m :  res] +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
872	`rejection_sampling_loss_term =>`	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
873	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
874	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
875	`(A, RealSigningPaperSimSampler(H))).main() @ &m :  res] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
876	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
877	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
878	`(A, ROExactHyperballPaperSimSampler(H))).main() @ &m :  res] +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
879	`rejection_sampling_loss_term.`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
880	`proof.`	Begins the proof script for the preceding proposition.
881	`move=> ro_attempt_hop.`	Introduces universally quantified variables or hypotheses into the proof context.
882	`rewrite (rom_internal_nma_real_signing_ro_attempt_paper_sim_exact H A &m).`	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
883	`by apply ro_attempt_hop.`	Discharges the current goal in one step using the cited simplification, lemma, or automation.
884	`qed.`	Ends the proof; EasyCrypt has accepted all remaining obligations.
885	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
886	`lemma real_signing_ro_exact_hyperball_hop_from_ro_signing_attempt_hop_loss`	States checked lemma real_signing_ro_exact_hyperball_hop_from_ro_signing_attempt_hop_loss. The following proof script must derive this proposition from prior definitions and lemmas.
887	`&m (loss :  real) :`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
888	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.

Line	EasyCrypt source	Mathematical reading
889	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
890	`(A, ROSigningAttemptPaperSimSampler(H))).main() @ &m :  res] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
891	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
892	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
893	`(A, ROExactHyperballPaperSimSampler(H))).main() @ &m :  res] +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
894	`loss =>`	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
895	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
896	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
897	`(A, RealSigningPaperSimSampler(H))).main() @ &m :  res] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
898	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
899	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
900	`(A, ROExactHyperballPaperSimSampler(H))).main() @ &m :  res] +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
901	`loss.`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
902	`proof.`	Begins the proof script for the preceding proposition.
903	`move=> ro_attempt_hop.`	Introduces universally quantified variables or hypotheses into the proof context.
904	`rewrite (rom_internal_nma_real_signing_ro_attempt_paper_sim_exact H A &m).`	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
905	`by apply ro_attempt_hop.`	Discharges the current goal in one step using the cited simplification, lemma, or automation.
906	`qed.`	Ends the proof; EasyCrypt has accepted all remaining obligations.
907	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
908	`lemma real_signing_ro_exact_hyperball_hop_from_budgeted_lifting &m :`	States checked lemma real_signing_ro_exact_hyperball_hop_from_budgeted_lifting. The following proof script must derive this proposition from prior definitions and lemmas.
909	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
910	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
911	`(A, ROSigningAttemptPaperSimSampler(H))).main() @ &m :  res] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
912	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
913	`ROMInternalTranscriptBudgetedPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
914	`(A, ROSigningAttemptPaperSimSampler(H))).main() @ &m :  res] =>`	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
915	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
916	`ROMInternalTranscriptBudgetedPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
917	`(A, ROExactHyperballPaperSimSampler(H))).main() @ &m :  res] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
918	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
919	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
920	`(A, ROExactHyperballPaperSimSampler(H))).main() @ &m :  res] =>`	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
921	`structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>`	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.

Line	EasyCrypt source	Mathematical reading
922	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
923	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
924	`(A, RealSigningPaperSimSampler(H))).main() @ &m :  res] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
925	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
926	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
927	`(A, ROExactHyperballPaperSimSampler(H))).main() @ &m :  res] +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
928	`rom_signature_query_budget * rejection_sampling_loss_term.`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
929	`proof.`	Begins the proof script for the preceding proposition.
930	`move=> attempt_unbudgeted_to_budgeted exact_budgeted_to_unbudgeted`	Introduces universally quantified variables or hypotheses into the proof context.
931	`sample_loss.`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
932	`apply`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
933	`(real_signing_ro_exact_hyperball_hop_from_ro_signing_attempt_hop_loss`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
934	`&m (rom_signature_query_budget * rejection_sampling_loss_term)).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
935	`apply`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
936	`(rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_budgeted_lifting`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
937	`H A &m).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
938	`+ by apply attempt_unbudgeted_to_budgeted.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
939	`+ by apply exact_budgeted_to_unbudgeted.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
940	`by apply sample_loss.`	Discharges the current goal in one step using the cited simplification, lemma, or automation.
941	`qed.`	Ends the proof; EasyCrypt has accepted all remaining obligations.
942	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
943	`lemma haetae_rejection_ro_exact_hyperball_sampling_hop_from_budgeted_lifting`	States checked lemma haetae_rejection_ro_exact_hyperball_sampling_hop_from_budgeted_lifting. The following proof script must derive this proposition from prior definitions and lemmas.
944	`&m :`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
945	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
946	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
947	`(A, ROSigningAttemptPaperSimSampler(H))).main() @ &m :  res] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
948	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
949	`ROMInternalTranscriptBudgetedPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
950	`(A, ROSigningAttemptPaperSimSampler(H))).main() @ &m :  res] =>`	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
951	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
952	`ROMInternalTranscriptBudgetedPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
953	`(A, ROExactHyperballPaperSimSampler(H))).main() @ &m :  res] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
954	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.

Line	EasyCrypt source	Mathematical reading
955	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
956	`(A, ROExactHyperballPaperSimSampler(H))).main() @ &m :  res] =>`	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
957	`structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>`	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
958	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
959	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
960	`(A, HAETAERejectionPaperSimSampler(H))).main() @ &m :  res] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
961	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
962	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
963	`(A, ROExactHyperballPaperSimSampler(H))).main() @ &m :  res] +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
964	`rom_signature_query_budget * rejection_sampling_loss_term.`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
965	`proof.`	Begins the proof script for the preceding proposition.
966	`move=> attempt_unbudgeted_to_budgeted exact_budgeted_to_unbudgeted`	Introduces universally quantified variables or hypotheses into the proof context.
967	`sample_loss.`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
968	`rewrite -(rom_internal_nma_real_signing_haetae_rejection_paper_sim_exact`	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
969	`H A &m).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
970	`by apply`	Discharges the current goal in one step using the cited simplification, lemma, or automation.
971	`(real_signing_ro_exact_hyperball_hop_from_budgeted_lifting &m).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
972	`qed.`	Ends the proof; EasyCrypt has accepted all remaining obligations.
973	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
974	`lemma euf_cma_security_from_exact_hyperball_paper_sim_bridge_and_counted_bimodal &m :`	States checked lemma euf_cma_security_from_exact_hyperball_paper_sim_bridge_and_counted_bimodal. The following proof script must derive this proposition from prior definitions and lemmas.
975	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
976	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
977	`(A, HAETAERejectionPaperSimSampler(H))).main() @ &m :  res] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
978	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
979	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
980	`(A, ExactHyperballHAETAERejectionPaperSimSampler(H))).main()`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
981	`@ &m :  res] =>`	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
982	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
983	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
984	`(A, ExactHyperballPaperSimSampler(H))).main() @ &m :  res] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
985	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
986	`Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m :  res] =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
987	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] <= mlwe_hardness_term =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.

Line	EasyCrypt source	Mathematical reading
988	`Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
989	`@ &m :  res] <= module_sis_hardness_term =>`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
990	`Pr[SIG.EUF_CMA(H, HAETAE, A).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
991	`haetae_euf_counted_rom_bound.`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
992	`proof.`	Begins the proof script for the preceding proposition.
993	`move=> exact_bridge exact_nma_hop mlwe_bound msis_bound.`	Introduces universally quantified variables or hypotheses into the proof context.
994	`apply (euf_cma_security_from_haetae_rejection_paper_sim_nma_and_counted_bimodal`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
995	`&m).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
996	`+ apply (ler_trans`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
997	`(Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
998	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
999	`(A, ExactHyperballHAETAERejectionPaperSimSampler(H))).main()`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1000	`@ &m :  res])).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1001	`+ by apply exact_bridge.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1002	`+ by apply (exact_hyperball_haetae_rejection_nma_bound_from_paper_sim &m).`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1003	`+ by apply mlwe_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1004	`+ by apply msis_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1005	`qed.`	Ends the proof; EasyCrypt has accepted all remaining obligations.
1006	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1007	`lemma euf_cma_security_from_exact_hyperball_sampling_hop_and_counted_bimodal &m :`	States checked lemma euf_cma_security_from_exact_hyperball_sampling_hop_and_counted_bimodal. The following proof script must derive this proposition from prior definitions and lemmas.
1008	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1009	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1010	`(A, HAETAERejectionPaperSimSampler(H))).main() @ &m :  res] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1011	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1012	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1013	`(A, ExactHyperballHAETAERejectionPaperSimSampler(H))).main()`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1014	`@ &m :  res] + rejection_sampling_loss_term =>`	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1015	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1016	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1017	`(A, ExactHyperballPaperSimSampler(H))).main() @ &m :  res] +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1018	`rejection_sampling_loss_term <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1019	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1020	`Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m :  res] =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.

Line	EasyCrypt source	Mathematical reading
1021	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] <= mlwe_hardness_term =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1022	`Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1023	`@ &m :  res] <= module_sis_hardness_term =>`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1024	`Pr[SIG.EUF_CMA(H, HAETAE, A).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1025	`haetae_euf_counted_rom_bound.`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1026	`proof.`	Begins the proof script for the preceding proposition.
1027	`move=> sampling_hop exact_nma_hop mlwe_bound msis_bound.`	Introduces universally quantified variables or hypotheses into the proof context.
1028	`apply (euf_cma_security_from_haetae_rejection_paper_sim_nma_and_counted_bimodal`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1029	`&m).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1030	`+ by apply (haetae_rejection_nma_bound_from_exact_hyperball_sampling_hop`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1031	`&m).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1032	`+ by apply mlwe_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1033	`+ by apply msis_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1034	`qed.`	Ends the proof; EasyCrypt has accepted all remaining obligations.
1035	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1036	`lemma euf_cma_security_from_ro_exact_hyperball_sampling_hop_and_counted_bimodal &m :`	States checked lemma euf_cma_security_from_ro_exact_hyperball_sampling_hop_and_counted_bimodal. The following proof script must derive this proposition from prior definitions and lemmas.
1037	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1038	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1039	`(A, RealSigningPaperSimSampler(H))).main() @ &m :  res] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1040	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1041	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1042	`(A, ROExactHyperballPaperSimSampler(H))).main() @ &m :  res] +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1043	`rejection_sampling_loss_term =>`	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1044	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1045	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1046	`(A, ROExactHyperballPaperSimSampler(H))).main() @ &m :  res] +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1047	`rejection_sampling_loss_term <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1048	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1049	`Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m :  res] =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1050	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] <= mlwe_hardness_term =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1051	`Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1052	`@ &m :  res] <= module_sis_hardness_term =>`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1053	`Pr[SIG.EUF_CMA(H, HAETAE, A).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.

Line	EasyCrypt source	Mathematical reading
1054	`haetae_euf_counted_rom_bound.`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1055	`proof.`	Begins the proof script for the preceding proposition.
1056	`move=> real_ro_exact_hop ro_exact_nma_hop mlwe_bound msis_bound.`	Introduces universally quantified variables or hypotheses into the proof context.
1057	`apply (euf_cma_security_from_haetae_rejection_paper_sim_nma_and_counted_bimodal`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1058	`&m).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1059	`+ apply (ler_trans`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1060	`(Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1061	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1062	`(A, ROExactHyperballPaperSimSampler(H))).main() @ &m :  res] +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1063	`rejection_sampling_loss_term)).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1064	`+ by apply`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1065	`(haetae_rejection_ro_exact_hyperball_sampling_hop_from_real_signing_ro_exact_hyperball_hop`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1066	`&m).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1067	`+ by apply ro_exact_nma_hop.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1068	`+ by apply mlwe_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1069	`+ by apply msis_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1070	`qed.`	Ends the proof; EasyCrypt has accepted all remaining obligations.
1071	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1072	`lemma euf_cma_security_from_ro_signing_attempt_to_ro_exact_hyperball_hop_and_counted_bimodal &m :`	States checked lemma euf_cma_security_from_ro_signing_attempt_to_ro_exact_hyperball_hop_and_counted_ The following proof script must derive this proposition from prior definitions and lemmas.
1073	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1074	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1075	`(A, ROSigningAttemptPaperSimSampler(H))).main() @ &m :  res] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1076	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1077	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1078	`(A, ROExactHyperballPaperSimSampler(H))).main() @ &m :  res] +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1079	`rejection_sampling_loss_term =>`	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1080	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1081	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1082	`(A, ROExactHyperballPaperSimSampler(H))).main() @ &m :  res] +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1083	`rejection_sampling_loss_term <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1084	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1085	`Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m :  res] =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1086	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] <= mlwe_hardness_term =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.

Line	EasyCrypt source	Mathematical reading
1087	`Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1088	`@ &m :  res] <= module_sis_hardness_term =>`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1089	`Pr[SIG.EUF_CMA(H, HAETAE, A).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1090	`haetae_euf_counted_rom_bound.`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1091	`proof.`	Begins the proof script for the preceding proposition.
1092	`move=> ro_attempt_hop ro_exact_nma_hop mlwe_bound msis_bound.`	Introduces universally quantified variables or hypotheses into the proof context.
1093	`apply (euf_cma_security_from_ro_exact_hyperball_sampling_hop_and_counted_bimodal`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1094	`&m).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1095	`+ by apply`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1096	`(real_signing_ro_exact_hyperball_hop_from_ro_signing_attempt_hop &m).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1097	`+ by apply ro_exact_nma_hop.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1098	`+ by apply mlwe_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1099	`+ by apply msis_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1100	`qed.`	Ends the proof; EasyCrypt has accepted all remaining obligations.
1101	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1102	`lemma real_signing_ro_exact_hyperball_hop_checked_from_loss_budget &m :`	States checked lemma real_signing_ro_exact_hyperball_hop_checked_from_loss_budget. The following proof script must derive this proposition from prior definitions and lemmas.
1103	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1104	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1105	`(A, RealSigningPaperSimSampler(H))).main() @ &m :  res] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1106	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1107	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1108	`(A, ROExactHyperballPaperSimSampler(H))).main() @ &m :  res] +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1109	`rejection_sampling_loss_term.`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1110	`proof.`	Begins the proof script for the preceding proposition.
1111	`apply (real_signing_ro_exact_hyperball_hop_from_ro_signing_attempt_hop &m).`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1112	`by apply (rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_bound`	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1113	`H A &m).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1114	`qed.`	Ends the proof; EasyCrypt has accepted all remaining obligations.
1115	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1116	`lemma real_signing_ro_exact_hyperball_hop_from_paper_sample_lifting &m :`	States checked lemma real_signing_ro_exact_hyperball_hop_from_paper_sample_lifting. The following proof script must derive this proposition from prior definitions and lemmas.
1117	`structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>`	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1118	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1119	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
1120	`(A, ROSigningAttemptPaperSimSampler(H))).main() @ &m :  res] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1121	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1122	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1123	`(A, ROExactHyperballPaperSimSampler(H))).main() @ &m :  res] +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1124	`rejection_sampling_loss_term =>`	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1125	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1126	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1127	`(A, RealSigningPaperSimSampler(H))).main() @ &m :  res] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1128	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1129	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1130	`(A, ROExactHyperballPaperSimSampler(H))).main() @ &m :  res] +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1131	`rejection_sampling_loss_term.`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1132	`proof.`	Begins the proof script for the preceding proposition.
1133	`move=> sample_loss lifted_loss.`	Introduces universally quantified variables or hypotheses into the proof context.
1134	`apply (real_signing_ro_exact_hyperball_hop_from_ro_signing_attempt_hop &m).`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1135	`by apply`	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1136	`(rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_paper_sample_lifting`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1137	`H A &m).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1138	`qed.`	Ends the proof; EasyCrypt has accepted all remaining obligations.
1139	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1140	`lemma euf_cma_security_from_paper_sample_lifting_and_counted_bimodal &m :`	States checked lemma euf_cma_security_from_paper_sample_lifting_and_counted_bimodal. The following proof script must derive this proposition from prior definitions and lemmas.
1141	`structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>`	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1142	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1143	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1144	`(A, ROSigningAttemptPaperSimSampler(H))).main() @ &m :  res] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1145	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1146	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1147	`(A, ROExactHyperballPaperSimSampler(H))).main() @ &m :  res] +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1148	`rejection_sampling_loss_term =>`	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1149	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1150	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1151	`(A, ROExactHyperballPaperSimSampler(H))).main() @ &m :  res] +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1152	`rejection_sampling_loss_term <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.

Line	EasyCrypt source	Mathematical reading
1153	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1154	`Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m :  res] =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1155	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] <= mlwe_hardness_term =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1156	`Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1157	`@ &m :  res] <= module_sis_hardness_term =>`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1158	`Pr[SIG.EUF_CMA(H, HAETAE, A).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1159	`haetae_euf_counted_rom_bound.`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1160	`proof.`	Begins the proof script for the preceding proposition.
1161	`move=> sample_loss lifted_loss ro_exact_nma_hop mlwe_bound msis_bound.`	Introduces universally quantified variables or hypotheses into the proof context.
1162	`apply (euf_cma_security_from_ro_exact_hyperball_sampling_hop_and_counted_bimodal`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1163	`&m).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1164	`+ by apply (real_signing_ro_exact_hyperball_hop_from_paper_sample_lifting &m).`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1165	`+ by apply ro_exact_nma_hop.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1166	`+ by apply mlwe_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1167	`+ by apply msis_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1168	`qed.`	Ends the proof; EasyCrypt has accepted all remaining obligations.
1169	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1170	`lemma euf_cma_security_from_budgeted_paper_sample_lifting_and_counted_bimodal`	States checked lemma euf_cma_security_from_budgeted_paper_sample_lifting_and_counted_bimodal. The following proof script must derive this proposition from prior definitions and lemmas.
1171	`&m :`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1172	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1173	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1174	`(A, ROSigningAttemptPaperSimSampler(H))).main() @ &m :  res] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1175	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1176	`ROMInternalTranscriptBudgetedPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1177	`(A, ROSigningAttemptPaperSimSampler(H))).main() @ &m :  res] =>`	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1178	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1179	`ROMInternalTranscriptBudgetedPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1180	`(A, ROExactHyperballPaperSimSampler(H))).main() @ &m :  res] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1181	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1182	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1183	`(A, ROExactHyperballPaperSimSampler(H))).main() @ &m :  res] =>`	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1184	`structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>`	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1185	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.

Line	EasyCrypt source	Mathematical reading
1186	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1187	`(A, ROExactHyperballPaperSimSampler(H))).main() @ &m :  res] +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1188	`rom_signature_query_budget * rejection_sampling_loss_term <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1189	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1190	`Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m :  res] =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1191	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] <= mlwe_hardness_term =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1192	`Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1193	`@ &m :  res] <= module_sis_hardness_term =>`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1194	`Pr[SIG.EUF_CMA(H, HAETAE, A).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1195	`haetae_euf_counted_rom_bound.`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1196	`proof.`	Begins the proof script for the preceding proposition.
1197	`move=> attempt_unbudgeted_to_budgeted exact_budgeted_to_unbudgeted`	Introduces universally quantified variables or hypotheses into the proof context.
1198	`sample_loss ro_exact_nma_hop mlwe_bound msis_bound.`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1199	`apply (euf_cma_security_from_haetae_rejection_paper_sim_nma_and_counted_bimodal`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1200	`&m).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1201	`+ apply (ler_trans`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1202	`(Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1203	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1204	`(A, ROExactHyperballPaperSimSampler(H))).main() @ &m :  res] +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1205	`rom_signature_query_budget * rejection_sampling_loss_term)).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1206	`+ apply`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1207	`(haetae_rejection_ro_exact_hyperball_sampling_hop_from_budgeted_lifting`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1208	`&m).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1209	`+ by apply attempt_unbudgeted_to_budgeted.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1210	`+ by apply exact_budgeted_to_unbudgeted.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1211	`by apply sample_loss.`	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1212	`by apply ro_exact_nma_hop.`	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1213	`+ by apply mlwe_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1214	`by apply msis_bound.`	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1215	`qed.`	Ends the proof; EasyCrypt has accepted all remaining obligations.
1216	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1217	`lemma euf_cma_security_from_checked_ro_sampling_hop_and_counted_bimodal &m :`	States checked lemma euf_cma_security_from_checked_ro_sampling_hop_and_counted_bimodal. The following proof script must derive this proposition from prior definitions and lemmas.
1218	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.

Line	EasyCrypt source	Mathematical reading
1219	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1220	`(A, ROExactHyperballPaperSimSampler(H))).main() @ &m :  res] +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1221	`rejection_sampling_loss_term <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1222	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1223	`Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m :  res] =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1224	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] <= mlwe_hardness_term =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1225	`Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1226	`@ &m :  res] <= module_sis_hardness_term =>`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1227	`Pr[SIG.EUF_CMA(H, HAETAE, A).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1228	`haetae_euf_counted_rom_bound.`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1229	`proof.`	Begins the proof script for the preceding proposition.
1230	`move=> ro_exact_nma_hop mlwe_bound msis_bound.`	Introduces universally quantified variables or hypotheses into the proof context.
1231	`apply (euf_cma_security_from_ro_exact_hyperball_sampling_hop_and_counted_bimodal`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1232	`&m).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1233	`+ by apply (real_signing_ro_exact_hyperball_hop_checked_from_loss_budget &m).`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1234	`+ by apply ro_exact_nma_hop.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1235	`+ by apply mlwe_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1236	`+ by apply msis_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1237	`qed.`	Ends the proof; EasyCrypt has accepted all remaining obligations.
1238	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1239	`lemma euf_cma_security_from_real_signing_exact_hyperball_hop_and_counted_bimodal &m :`	States checked lemma euf_cma_security_from_real_signing_exact_hyperball_hop_and_counted_bimodal. The following proof script must derive this proposition from prior definitions and lemmas.
1240	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1241	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1242	`(A, RealSigningPaperSimSampler(H))).main() @ &m :  res] <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1243	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1244	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1245	`(A, ExactHyperballPaperSimSampler(H))).main() @ &m :  res] +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1246	`rejection_sampling_loss_term =>`	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1247	`Pr[SIG.UF_NMA(H, HAETAE,`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1248	`ROMInternalTranscriptPaperSimAsNMA`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1249	`(A, ExactHyperballPaperSimSampler(H))).main() @ &m :  res] +`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1250	`rejection_sampling_loss_term <=`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1251	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] +`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.

Line	EasyCrypt source	Mathematical reading
1252	`Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m :  res] =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1253	`Pr[MLWE_Game(Bmlwe).main() @ &m :  res] <= mlwe_hardness_term =>`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1254	`Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1255	`@ &m :  res] <= module_sis_hardness_term =>`	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1256	`Pr[SIG.EUF_CMA(H, HAETAE, A).main() @ &m :  res] <=`	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1257	`haetae_euf_counted_rom_bound.`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1258	`proof.`	Begins the proof script for the preceding proposition.
1259	`move=> real_exact_hop exact_nma_hop mlwe_bound msis_bound.`	Introduces universally quantified variables or hypotheses into the proof context.
1260	`apply (euf_cma_security_from_exact_hyperball_sampling_hop_and_counted_bimodal`	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1261	`&m).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1262	`+ by apply`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1263	`(haetae_rejection_exact_hyperball_sampling_hop_from_real_signing_exact_hyperball_hop`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1264	`&m).`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1265	`+ by apply exact_nma_hop.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1266	`+ by apply mlwe_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1267	`+ by apply msis_bound.`	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1268	`qed.`	Ends the proof; EasyCrypt has accepted all remaining obligations.
1269	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1270	`end section MechanizedROMTranscriptConstructedNMALift.`	Closes the current section, discharging local module and variable scope.
1271	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1272	`end HAETAE_Security.`	Closes the current theory or module block.